

Module 1: JDBC

- Introduction to JDBC
- JDBC Architecture
- JDBC Drivers (Type 1, 2, 3, 4)
- DriverManager
- Connection
- Statement
- PreparedStatement
- CallableStatement
- ResultSet
- CRUD Operations
- Parameterized Queries
- DAO Pattern
- Try-with-Resources

Module 2: Jakarta EE (Java EE)

- Web Applications
- Enterprise Applications
- Jakarta EE Architecture
- Managed vs Unmanaged Components
- Web Container
- EJB Container
- Web Server vs Application Server
- Tomcat, GlassFish, JBoss, WildFly

Module 3: Servlets

- Servlet Basics
- Servlet API
- HttpServlet
- Servlet Lifecycle
 - init()
 - service()
 - destroy()
- doGet()
- doPost()
- HTML Form Processing
- Request Parameters
- RequestDispatcher

- forward()
- include()

Module 4: Session Management

- Stateless vs Stateful Protocols
- Session Tracking Techniques
 - URL Rewriting
 - Hidden Fields
 - Cookies
 - HttpSession
- Session ID
- Session Lifecycle

Module 5: JSP (Java Server Pages)

- JSP Basics
- JSP Lifecycle
- JSP File Structure
- JSP Directives
 - page
 - include
 - taglib
- JSP Scripting Elements
 - Declaration
 - Scriptlet
 - Expression
- JSP Implicit Objects
 - request
 - response
 - session
 - application
 - pageContext
 - exception

Module 6: JSP Standard Actions

- `<jsp:useBean>`
- `<jsp:setProperty>`
- `<jsp:getProperty>`
- `<jsp:forward>`
- `<jsp:include>`

- `<jsp:param>`
- Bean Scopes
 - page
 - request
 - session
 - application

Module 7: Hibernate

- ORM Concepts
- Hibernate Architecture
- Hibernate Core API
 - Session
 - SessionFactory
 - Configuration
 - Transaction
 - Query
- Mapping Metadata
- XML Mapping
- Annotation Mapping
- JPA Annotations
 - @Entity
 - @Table
 - @Id
 - @Column
- CRUD Operations
- Entity Lifecycle
- HQL (Hibernate Query Language)
- Constructor Expression

Module 8: Spring Core

- Spring Framework Introduction
- Spring Architecture
- Spring Modules
- Bean Management
- Dependency Injection (DI)
- IoC (Inversion of Control)
- ApplicationContext
- XML Configuration
- Java Configuration
- Annotations
 - @Configuration

- @Bean
- @Autowired
- @Lazy
- @Scope
- @Primary
- @Qualifier
- @Component
- @ComponentScan

Module 9: Spring MVC

- MVC Architecture
- MVC1 vs MVC2
- Front Controller Pattern
- DispatcherServlet
- Controller
- Model
- View
- View Resolver
- @Controller
- @RequestMapping
- @RequestParam
- Model Interface
- @SessionAttributes

Module 10: Spring Boot

- Spring Boot Introduction
- Spring Initializr
- Embedded Tomcat
- @SpringBootApplication
- SpringApplication
- application.properties
- Spring Boot MVC Setup

Module 11: REST API

- Web Services
- SOAP vs REST
- REST Architecture
- REST Endpoints
- JSON

- @RestController
- @ResponseBody
- Layered Architecture
 - Controller Layer
 - Service Layer
 - DAO Layer
- @Service
- @Repository

Most Important Topics for CDAC Placements

1. Core Java & Collections
2. JDBC
3. Servlets
4. JSP
5. Session Management
6. Hibernate + JPA
7. Spring Core
8. Dependency Injection
9. Spring MVC
10. Spring Boot
11. REST APIs
12. MySQL Queries

Day 1-2: JDBC

What is JDBC?

- JDBC = Java Database Connectivity
- Used to connect Java applications with databases.

JDBC Steps

1. Load Driver
2. Get Connection
3. Create Statement
4. Execute Query
5. Process ResultSet
6. Close Resources

Important Interfaces

- DriverManager

- Connection
- Statement
- PreparedStatement
- ResultSet

Statement vs PreparedStatement

Statement	PreparedStatement
Simple query	Parameterized query
Less secure	Prevents SQL Injection
Slower	Faster

CRUD Methods

executeQuery() // SELECT

executeUpdate() // INSERT, UPDATE, DELETE

DAO Pattern

- DAO = Data Access Object
 - Separates database code from business logic.
-

Day 2: Jakarta EE

Web Application

- Accessed through browser.

Enterprise Application

Provides:

- Security
- Scalability
- Transactions
- Persistence

Servers

Web Server

- Apache Tomcat
- Jetty

Application Server

- GlassFish
- JBoss
- WebLogic

Interview Question

Difference between Web Server and Application Server?

Application Server provides both Web Container and EJB Container.

Day 2-3: Servlets

Servlet

- Server-side Java program.
- Generates dynamic web content.

Servlet Lifecycle

init()

service()

destroy()

HTTP Methods

doGet()

doPost()

Difference

GET

POST

Data in URL Data in Body

Less Secure More Secure

RequestDispatcher

forward()
include()

Day 3: Session Management

Why?

HTTP is Stateless.

Session Tracking Techniques

1. URL Rewriting
2. Hidden Fields
3. Cookies
4. HttpSession

HttpSession Methods

setAttribute()
getAttribute()
removeAttribute()
invalidate()

Interview Question

Difference between Cookie and Session?

Cookie → Client Side

Session → Server Side

Day 3-4: JSP

JSP

- Java Server Pages
- Used for UI/Presentation Layer

JSP Lifecycle

jspInit()
_jspService()
jspDestroy()

JSP Directives

<%@ page %>
<%@ include %>
<%@ taglib %>

Scripting Elements

<%! %> // Declaration
<% %> // Scriptlet
<%= %> // Expression

Implicit Objects

request
response
session
application
out
pageContext
exception

Hibernate

What is ORM?

ORM = Object Relational Mapping

Maps:

Java Object <--> Database Table

Advantages

- Less SQL
- Easy CRUD
- Transaction Management
- Caching

Hibernate Core API

Session
SessionFactory
Configuration
Transaction
Query

Important Annotations

@Entity
@Table
@Id
@Column

CRUD Methods

save()
find()
update()
remove()

Entity Lifecycle

Transient
Persistent
Detached
Removed

HQL

Hibernate Query Language

```
FROM Student  
SELECT s.name FROM Student s
```

Spring Core

Spring Framework

Provides:

- Dependency Injection
- Bean Management

- Security
- MVC
- REST

Bean

Any object managed by Spring.

Dependency Injection (DI)

Spring creates objects and injects dependencies automatically.

Important Annotations

@Configuration
@Bean
@Component
@ComponentScan
@Autowired
@Primary
@Qualifier
@Scope
@Lazy

Bean Scope

singleton (default)
prototype
request
session

Spring MVC

MVC

M = Model

V = View

C = Controller

Components

DispatcherServlet

Controller
Model
View
ViewResolver

Important Annotations

@Controller
@RequestMapping
@RequestParam
@SessionAttributes

Flow

Browser
↓
DispatcherServlet
↓
Controller
↓
Model
↓
View(JSP)

Spring Boot

Why Spring Boot?

- Less Configuration
- Embedded Tomcat
- Faster Development

Important Classes

@SpringBootApplication
SpringApplication.run()

Default Port

8080

Change in:

server.port=9090

REST API

REST

Representational State Transfer

HTTP Methods

GET -> Read

POST -> Create

PUT -> Update

DELETE -> Delete

Data Format

```
{  
  "id": 1,  
  "name": "Rohit"  
}
```

Important Annotations

@RestController

@ResponseBody

@Service

@Repository

REST Layers

Controller Layer

↓

Service Layer

↓

DAO Layer

↓

Database

1. JDBC

Definition

JDBC (Java Database Connectivity) is an API that allows Java applications to communicate with databases.

Example

```
Connection con =  
DriverManager.getConnection(  
"jdbc:mysql://localhost:3306/studentdb",  
"root",  
"root");
```

Remember

👉 JDBC = Bridge between Java and Database

2. Driver

Definition

A Driver converts JDBC commands into database-specific commands.

Example

```
com.mysql.cj.jdbc.Driver
```

Remember

👉 Driver = Translator between Java and MySQL

3. Connection

Definition

A Connection represents a session between Java application and database.

Example

```
Connection con = DriverManager.getConnection(...);
```

Remember

👉 Connection = Phone call with database

4. Statement

Definition

Used to execute simple SQL queries.

Example

```
Statement st = con.createStatement();  
ResultSet rs = st.executeQuery("select * from student");
```

Remember

👉 Statement = Fixed SQL

5. PreparedStatement

Definition

Used to execute parameterized SQL queries.

Example

```
PreparedStatement ps =  
con.prepareStatement(  
"select * from student where id=?");  
  
ps.setInt(1,101);
```

Remember

👉 PreparedStatement = Secure SQL

6. ResultSet

Definition

Stores data returned by SELECT query.

Example

```
while(rs.next())
{
    System.out.println(rs.getString("name"));
}
```

Remember

👉 ResultSet = Table in Java

7. DAO

Definition

DAO (Data Access Object) separates database code from business logic.

Example

```
StudentDao
StudentDaoImpl
```

Remember

👉 DAO = Database Manager Class

8. Servlet

Definition

A Servlet is a Java class that handles web requests and generates responses.

Example

```
public class LoginServlet
extends HttpServlet
{
}
```

Remember

👉 Servlet = Java program running on server

9. Servlet Life Cycle

Definition

Stages through which a servlet passes.

Methods

```
init()
service()
destroy()
```

Example

```
public void init()
{
    System.out.println("Loaded");
}
```

Remember

Create → Serve → Destroy

10. GET Request

Definition

Sends data through URL.

Example

```
<form method="get">
```

URL

login?id=101

Remember

👉 Data visible in URL

11. POST Request

Definition

Sends data through request body.

Example

```
<form method="post">
```

Remember

👉 Used for passwords

12. RequestDispatcher

Definition

Used to transfer control from one resource to another.

Example

```
RequestDispatcher rd=  
request.getRequestDispatcher("home.jsp");
```

```
rd.forward(request,response);
```

Remember

👉 Dispatcher = Traffic Police

13. Session

Definition

Stores user information across multiple requests.

Example

```
HttpSession session=  
request.getSession();
```

```
session.setAttribute(  
"name","Rohit");
```

Remember

👉 Session = User Memory

14. Cookie

Definition

Small file stored on client browser.

Example

```
Cookie c =  
new Cookie("name","Rohit");
```

```
response.addCookie(c);
```

Remember

👉 Cookie = Browser Memory

15. JSP

Definition

JSP (Java Server Pages) is used to create dynamic web pages.

Example

```
<h1>Welcome</h1>
```

```
<%= "Hello Rohit" %>
```

Remember

👉 JSP = HTML + Java

16. JSP Directives

Definition

Special instructions for JSP Container.

Example

```
<%@ page %>
```

```
<%@ include %>
```

```
<%@ taglib %>
```

Remember

👉 Directive = Configuration

17. Scriptlet

Definition

Java code written inside JSP.

Example

```
<%  
int x=10;  
%>
```

Remember

👉 Scriptlet = Java block

18. Expression

Definition

Prints value directly.

Example

```
<%= x %>
```

Remember

👉 Expression = Display value

19. ORM

Definition

Object Relational Mapping maps Java objects to database tables.

Example

Student object



student table

Remember

👉 Object ↔ Table

20. Hibernate

Definition

Hibernate is an ORM framework.

Example

```
session.save(student);
```

Remember

👉 Hibernate = JDBC without much SQL

21. SessionFactory

Definition

Creates Hibernate Sessions.

Example

```
SessionFactory sf=  
cfg.buildSessionFactory();
```

Remember

👉 Factory creates Sessions

22. Session (Hibernate)

Definition

Used for CRUD operations.

Example

```
session.save(obj);
```

Remember

👉 Session = Database Worker

23. @Entity

Definition

Marks class as database entity.

Example

```
@Entity
public class Student
{
}
```

Remember

👉 Entity = Table Class

24. @Id

Definition

Primary Key.

Example

```
@Id  
private int id;
```

Remember

👉 @Id = Unique Identifier

25. Spring Framework

Definition

Framework for building enterprise applications.

Remember

👉 Spring manages objects automatically

26. Bean

Definition

Object managed by Spring.

Example

```
@Bean  
public Student student()  
{  
    return new Student();  
}
```

Remember

👉 Bean = Spring Object

27. Dependency Injection

Definition

Spring automatically provides required objects.

Example

```
@Autowired  
StudentDao dao;
```

Remember

👉 DI = Don't create, let Spring provide

28. @Autowired

Definition

Automatically injects dependency.

Example

```
@Autowired  
StudentService service;
```

Remember

👉 Auto Connect Objects

29. MVC

Definition

Model
View
Controller

Example

JSP → View

Servlet → Controller

Java Class → Model

Remember

👉 MVC = Separate responsibilities

30. Spring Boot

Definition

Extension of Spring that reduces configuration.

Example

```
@SpringBootApplication  
public class DemoApp  
{  
}
```

Remember

👉 Spring + Auto Configuration

31. REST API

Definition

Way to exchange data over HTTP.

Example

```
@GetMapping("/students")
```

Response

```
{  
  "id":101,  
  "name":"Rohit"  
}
```

Remember

👉 REST = Data Service

32. @RestController

Definition

Creates REST APIs.

Example

```
@RestController  
public class StudentController  
{  
}
```

Remember

👉 Controller returning JSON

33. @Service

Definition

Business Logic Layer.

Example

```
@Service
public class StudentService
{
}
```

Remember

👉 Service = Logic Layer

34. @Repository

Definition

Database Layer.

Example

```
@Repository
public class StudentDao
{
}
```

Remember

👉 Repository = Database Layer

One-Line Memory Trick for Entire Module

JDBC → Connect DB
Servlet → Handle Request
Session → Remember User
JSP → Show UI
Hibernate → ORM Framework
Spring → Manage Objects
MVC → Structure Application
Spring Boot → Fast Development
REST → Send JSON Data